Geavanceerde Computerarchitectuur
Session 3

Christoph Sanders & Wout Corijn

2025-02-24

1 Probleem

In dit labo worden twee beeldverwerkingstaken uitgevoerd op de GPU[1] aan de hand van Cuda.

1. **Beeldinversie & warps** – Implementeer een CUDA-kernel die een afbeelding inverteert volgens de transformatie:

$$x = 255 - x$$

   voor elke kleurcomponent (R, G, B). Er wordt geanalyseerd hoe het aantal threads (volledige of gedeeltelijke warps) de rekentijd beïnvloedt.

2. **Thread Divergentie & Gaussiaans blur** – De voorgaande kernel wordt uitgebreid zodat het rode kanaal (R) wordt vervaagd met een 1D Gaussiaans filter volgens:

$$r_i = 0.1r_{i-2} + 0.25r_{i-1} + 0.5r_i + 0.25r_{i+1} + 0.1r_{i+2}$$

   terwijl de groene (G) en blauwe (B) kanalen nog steeds worden geïnverteerd met $x = 255 - x$. Dit introduceert *thread divergentie*, waarvan de invloed op de rekentijd wordt geanalyseerd.

2 Methode

2.1 Part 1 - warp

Als er minder threads beschikbaar zijn dan het aantal pixels, moet elke thread meerdere pixels verwerken. Om te bepalen welke thread welke pixels verwerkt, wordt de volgende formule gebruikt:

```
for (int i = blockIdx.x * blockDim.x + threadIdx.x;
        i < size;
        i += blockDim.x * gridDim.x)
{}
```

Op deze manier zal i aanduiden welke pixels door een bepaalde thread verwerkt moet worden, zonder te botsen met andere threads.

2.2 Part 2 - Thread divergence

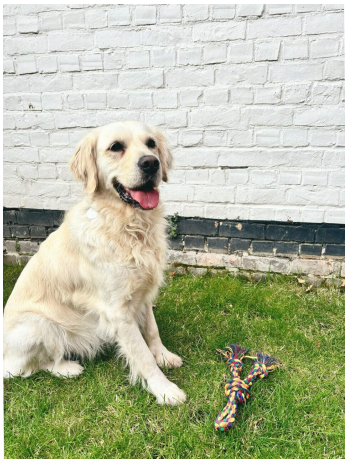
Bij thread divergence is het belangerijk om in te zien dat het herformatteren van de foto naar een andere structuur leidt tot een efficiënter gebruik van de GPU-resources.

In de oorspronkelijke indeling bevinden threads die blauwe, rode en groene pixels verwerken zich samen in een warp. Omdat de verwerkingstijd per kleur verschilt, waarbij blauwe en groene threads sneller verwerkt worden dan rode, moeten deze threads onnodig wachten op de afhandeling van de rode pixels binnen dezelfde warp.

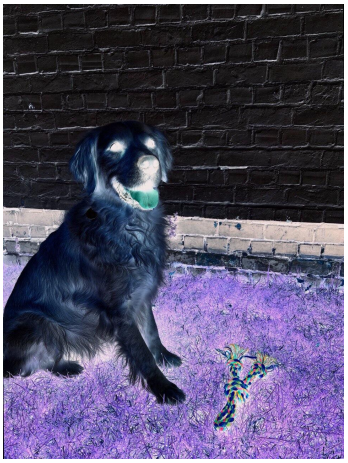
Door de foto anders te formatteren, worden threads gegroepeerd op basis van kleur. Een warp bestaat nu uit threads die dezelfde kleur verwerken, bijvoorbeeld alleen rode pixels. Hierdoor wordt de wachttijd binnen de warp geminimaliseerd, wat resulteert in een efficiëntere en snellere uitvoering.

[1] GPU: NVIDIA GeForce GTX 1050

3 Resultaat



(a) Normaal



(b) Inversie



(c) Inversie + blur

3.1 Part 1 - warp

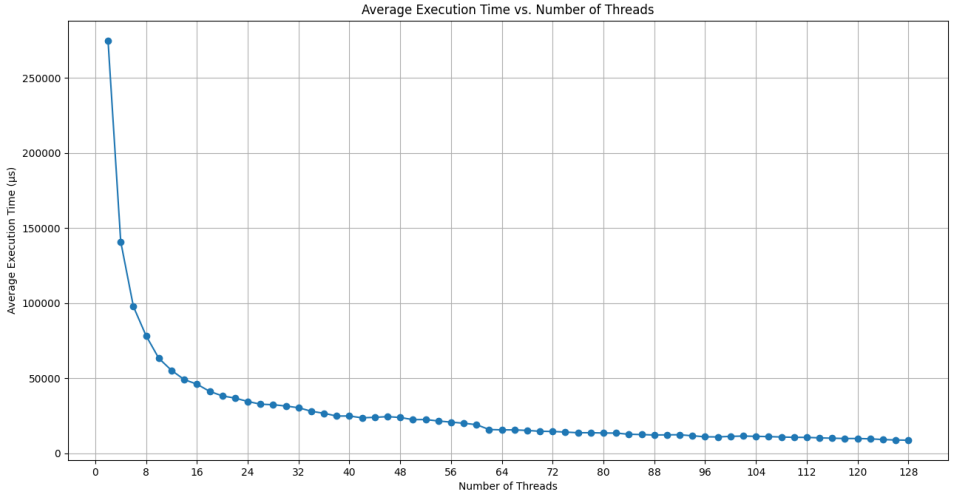
De onderstaande grafieken geven het gemiddelde weer van 10 tijdsmetingen in microseconden (µs) als functie van het aantal threads. Er wordt een onderscheid gemaakt tussen metingen waarbij de geheugenallocatie (malloc's) in CUDA inclusief en exclusief de tijdsmeting is gebeurd.

In de grafieken is zichtbaar dat de uitvoeringstijd exponentieel afneemt naarmate het aantal threads toeneemt. Bij een klein aantal threads is de werklast per thread groot. Wanneer meer threads worden toegevoegd, kan de GPU efficiënter werken door gebruik te maken van warps – groepen van 32 threads die tegelijkertijd worden uitgevoerd op CUDA cores.

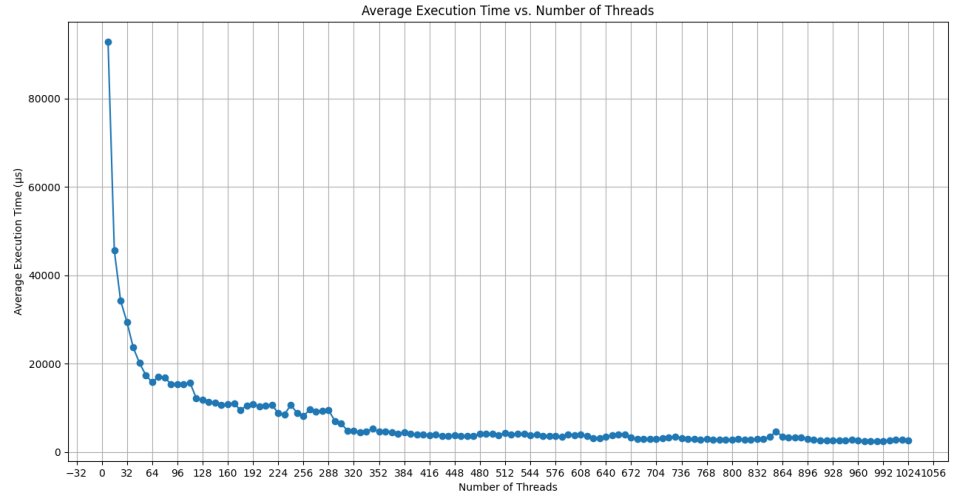
De uitvoeringstijd neemt niet volledig vloeiend af. Bij bepaalde punten is een kleine daling merkbaar. Dit gebeurt voornamelijk bij veelvouden van 32, wat overeenkomt met de grootte van een warp in CUDA. Wanneer het aantal threads een veelvoud van 32 bedraagt, kan de GPU optimaal gebruik maken van zijn rekeneenheden zonder dat er threads verloren gaan of inactief blijven.

Wanneer het aantal threads geen veelvoud van 32 is, kan een deel van de warps slechts gedeeltelijk gevuld zijn. Dit leidt tot inefficiëntie, aangezien een warp altijd in zijn geheel wordt uitgevoerd, zelfs als niet alle threads effectief nuttig werk verrichten.

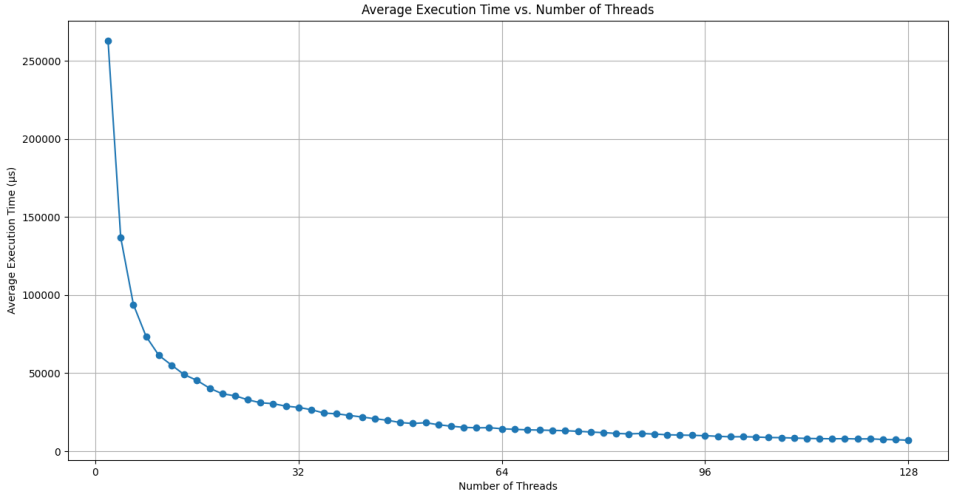
Grafiek 2 en 3 toont de tijdsmeting inclusief en grafiek 4 en 5 exclusief de memory allocatie in de meting.



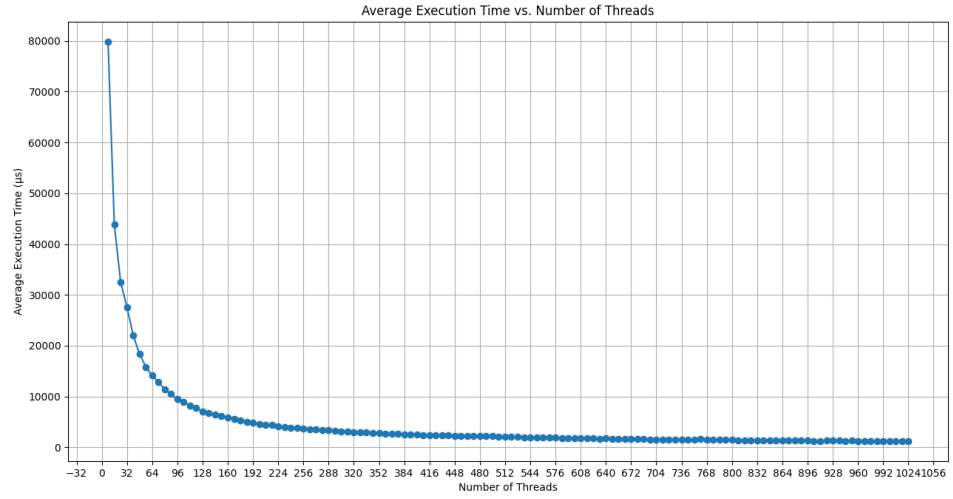
Grafiek 2: Varying thread size (incl. memory allocations)



Grafiek 3: Varying thread size (incl. memory allocations)



Grafiek 4: Varying thread size (excl. memory allocations)



Grafiek 5: Varying thread size (excl. memory allocations)

3.2 Part 2 - Thread divergence

Het herstructureren van de data heeft als voordeel dat wraps veel efficienter afgehandeld kunnen worden. In een zelfde wrap zit nu voornamelijk alleen threads die ofwel rode pixels ofwel blauwe/-groene pixels aan het verwerken zijn, hierdoor gaan er threads in een wrap minder lang op mekaar moeten wachten. En zal dit dus ook resulteren in een sneller programma.

4 Appendix

4.1 Warps

```
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <chrono>

#define M 960       // VR width
#define N 1280       // VR height
#define C 3         // Colors
#define OFFSET 16   // Header length


uint8_t* get_image_array(void){
    // CODE
}

void save_image_array(uint8_t* image_array){
    // CODE
}

__global__ void convertColor(uint8_t* image_array, int size) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
        i < size;
        i += blockDim.x * gridDim.x)
    {
        image_array[i] = 255 - image_array[i];
    }
}

int main (void) {

    // Read the image
    uint8_t* image_array = get_image_array();

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);

    int threads = 1024;
    uint8_t *d_image_array;
    int size = M*N*C;

    auto start = std::chrono::high_resolution_clock::now();

    cudaMalloc((void**)&d_image_array, size * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    convertColor<<<1, threads>>>(d_image_array, size);

    cudaMemcpy(new_image_array, d_image_array, size* sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    cudaFree(d_image_array);

    auto end = std::chrono::high_resolution_clock::now();

    auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end -
        start);
    std::cout << "Time␣difference:␣" << duration.count() << "␣microseconds" << std
        ::endl;

    // Save the image
    save_image_array(new_image_array);

    return 0;
}
```

4.2 Thread divergence (1)

```
#include <cstdint>      // Data types
#include <iostream>     // File operations
#include <chrono>

#define M 960        // VR width
#define N 1280        // VR height
#define C 3          // Colors
#define OFFSET 16    // Header length


uint8_t* get_image_array(void){
    // CODE
}

void save_image_array(uint8_t* image_array){
    // CODE
}

__global__ void convertColor(uint8_t* image_array,uint8_t* out_image_array, int
    size) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
        i < size;
        i += blockDim.x * gridDim.x)
    {
        if(i%3 || i < 6 || i > size-6){
            out_image_array[i] = 255 - image_array[i];
        }
        else{
            int i1 = i-6;
            int i2 = i-3;
            int i3 = i;
            int i4 = i+3;
            int i5 = i+6;
            out_image_array[i] = 0.1 * image_array[i1] + 0.25 * image_array[i2] +
                0.5 * image_array[i3] +
                0.25 * image_array[i4] + 0.5 + 0.1 * image_array[i5];
        }
    }
}

int main (void) {

    // Read the image
    uint8_t* image_array = get_image_array();

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);

    int threads = 1024;
    uint8_t *d_image_array;
    uint8_t *d_out_image_array;
    int size = M*N*C;

    auto start = std::chrono::high_resolution_clock::now();

    cudaMalloc((void**)&d_image_array, size * sizeof(uint8_t));
    cudaMalloc((void**)&d_out_image_array, size * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);
    cudaMemcpy(d_out_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    convertColor<<<1, threads>>>(d_image_array,d_out_image_array, size);

    cudaMemcpy(new_image_array, d_out_image_array, size* sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    cudaFree(d_image_array);
    cudaFree(d_out_image_array);
```

```
64
65      auto end = std::chrono::high_resolution_clock::now();
66
67      auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end -
            start);
68      std::cout << "Time difference: " << duration.count() << " microseconds" << std
            ::endl;
69
70      // Save the image
71      save_image_array(new_image_array);
72
73      return 0;
74  }
```

4.3 Thread divergence (2)

```
1   #include <cstdint>      // Data types
2   #include <iostream>     // File operations
3   #include <chrono>
4
5   #define M 960        // VR width
6   #define N 1280        // VR height
7   #define C 3          // Colors
8   #define OFFSET 16    // Header length
9
10
11  uint8_t* get_image_array(void){
12      // CODE
13  }
14
15
16  void save_image_array(uint8_t* image_array){
17      // CODE
18  }
19
20  __global__ void convertColor(uint8_t* image_array,uint8_t* out_image_array, int
        size) {
21      for (int i = blockIdx.x * blockDim.x + threadIdx.x;
22          i < size;
23          i += blockDim.x * gridDim.x)
24      {
25          if(i >= size/3){
26              out_image_array[i] = 255 - image_array[i];
27          }
28          else{
29              int i1 = i-6;
30              int i2 = i-3;
31              int i3 = i;
32              int i4 = i+3;
33              int i5 = i+6;
34              if(i1 < 0) i1 = 0;
35              if(i2 < 0) i2 = 0;
36              if(i4 > size/3) i4 = size/3;
37              if(i5 > size/3) i5 = size/3;
38              out_image_array[i] = 0.1 * image_array[i1] + 0.25 * image_array[i2] +
                    0.5 * image_array[i3] +
39                  0.25 * image_array[i4] + 0.5 + 0.1 * image_array[i5];
40          }
41
42      }
43  }
44
45  __global__ void reformat(uint8_t* image_array,uint8_t* out_image_array, int size) {
46      for (int i = blockIdx.x * blockDim.x + threadIdx.x;
47          i < size;
48          i += blockDim.x * gridDim.x)
49      {
50          if(i%3 == 0){
```

```
            // red
            out_image_array[i/3] = image_array[i];
        }
        else if(i%3 == 1){
            // green
            out_image_array[(i-1)/3 + size/3] = image_array[i];
        }
        else if(i%3 == 2){
            // blue
            out_image_array[(i-2)/3 + size*2/3] = image_array[i];
        }
    }
}

__global__ void deformat(uint8_t* image_array,uint8_t* out_image_array, int size) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x;
        i < size;
        i += blockDim.x * gridDim.x)
    {
        if(i < size/3){
            // red
            out_image_array[i*3] = image_array[i];
        }
        else if(i < size*2/3){
            // green
            int index = i - size/3;
            out_image_array[index*3 + 1] = image_array[i];
        }
        else if(i < size){
            // green
            int index = i - size*2/3;
            out_image_array[index*3 + 2] = image_array[i];
        }
    }
}

int main (void) {

    // Read the image
    uint8_t* image_array = get_image_array();

    // Allocate output
    uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);

    int threads = 1024;
    uint8_t *d_image_array;
    uint8_t *d_out_image_array;
    int size = M*N*C;

    auto start = std::chrono::high_resolution_clock::now();

    cudaMalloc((void**)&d_image_array, size * sizeof(uint8_t));
    cudaMalloc((void**)&d_out_image_array, size * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, size * sizeof(uint8_t),
        cudaMemcpyHostToDevice);

    reformat<<<1, threads>>>(d_image_array,d_out_image_array, size);

    convertColor<<<1, threads>>>(d_out_image_array, d_image_array, size);

    deformat<<<1, threads>>>(d_image_array,d_out_image_array, size);

    cudaMemcpy(new_image_array, d_out_image_array, size* sizeof(uint8_t),
        cudaMemcpyDeviceToHost);
    cudaFree(d_image_array);
    cudaFree(d_out_image_array);

    auto end = std::chrono::high_resolution_clock::now();

```

```
118     auto duration = std::chrono::duration_cast<std::chrono::microseconds>(end -
            start);
119     std::cout << "Time difference: " << duration.count() << " microseconds" << std
            ::endl;
120
121     // Save the image
122     save_image_array(new_image_array);
123
124     return 0;
125 }
```